

异构双 GPU 上的 HPCG 基准测试与负载均衡研究

基于 NVIDIA nvidia-hpcg 在 RTX 4090 + RTX 5080 工作站上的实验

实验日期: 2026-06-17

目录

1	引言	3
1.1	HPCG 基准测试概述	3
1.2	nvidia-hpcg 简介	3
1.3	项目目标	3
2	实现	4
2.1	对称双 GPU 配置	4
2.2	异构分区设计	4
2.3	关键 Bug 修复	5
2.3.1	1. extToLocMap 越界写入	5
2.3.2	2. 粗网格奇数维度导致 cuSPARSE SpSV 崩溃	5
2.3.3	3. CPU 亲和性修正	5
2.4	执行流程小结	5
3	结果分析	6
3.1	总体性能对比	6
3.2	单卡差异: RTX 5080 为何明显慢于 RTX 4090	7
3.3	对称双卡为何慢于单卡	8
3.4	异构分区的效果	9
3.5	规模与配置选择	9
4	不同维度分块对比实验	11
4.1	实验设计	11
4.2	实验结果	11
4.3	结果分析	12
4.3.1	对称分块差异	12
4.3.2	非对称分块收益	12
4.3.3	结论	12
5	RTX 5080 性能异常偏低的 PCIe 瓶颈分析	13
5.1	测试方法	13
5.2	PCIe 链路状态	13
5.2.1	空闲与满载对比	13
5.2.2	运行时 PCIe 宽度与吞吐	14
5.3	理论带宽估算	14
5.4	主板插槽规格	14
5.5	结论与建议	15
6	扩展性与运行时间分析	16
6.1	无 nsys 的完整五配置运行时间	16
6.2	更大规模的可行性实验	16

6.3 扩展性结论	16
-----------------	----

1 引言

1.1 HPCG 基准测试概述

HPCG (High Performance Conjugate Gradient) 是由美国 Sandia 国家实验室牵头设计的稀疏线性代数基准测试, 旨在补充 LINPACK/HPL 对稠密矩阵计算能力的评价, 从更接近实际科学与工程应用的角度衡量超级计算机的性能。HPCG 的核心计算模式是预处理共轭梯度法 (Preconditioned Conjugate Gradient, PCG), 其主要算子包括稀疏矩阵向量乘法 (SpMV)、向量点积 (DDOT)、向量更新 (WAXPY) 以及基于多层网格的几何粗化预处理。这些操作具有低计算密度、强内存依赖和频繁同步的特点, 因此对内存带宽、缓存效率、网络延迟以及节点内并行调度都提出了很高的要求。

HPCG 的评分以 **GFLOP/s** 为单位, 最终结果是多次运行后得到的稳定浮点性能。由于算法本身的局部性和全局通信需求, HPCG 在多 GPU、多节点场景下往往难以实现理想的线性扩展, 负载不均衡、通信开销以及同步等待会成为主要的性能瓶颈。

1.2 nvidia-hpcg 简介

nvidia-hpcg 是 NVIDIA 基于 HPCG 3.x 参考实现开发的 GPU 优化版本。它使用 CUDA、cuSPARSE、cuBLAS 等库将核心算子 offload 到 GPU, 并支持通过 MPI 在多个 GPU 之间进行分布式扩展。其默认执行模式 **GPUONLY** 会将所有计算任务放在 GPU 上, CPU 主要负责 MPI 通信控制和进程管理。

在本项目中, 我们基于 nvidia-hpcg 源码, 针对一台搭载 **RTX 4090** (CUDA 0, 快卡) 和 **RTX 5080** (CUDA 1, 慢卡) 的异构双 GPU 工作站进行了适配与扩展。目标是在保留 GPU 计算路径的前提下, 实现两块不同性能 GPU 的协同运行, 并通过非对称负载划分提升整体吞吐量。

1.3 项目目标

本项目围绕以下几个核心目标展开:

1. **编译与运行适配**: 在 RTX 4090 + RTX 5080 的混合架构上完成 nvidia-hpcg 的编译, 确保单卡与双卡模式均能正确运行并得到 VALID 结果。
2. **对称双卡基线**: 建立若干对称双 GPU 配置 (全局规模相同、每 rank 本地规模相同), 作为性能对比的基准。
3. **异构负载均衡**: 引入 `--het-split`、`--het-dim`、`--het-ratio` 等参数, 使快卡 (RTX 4090) 承担更多工作、慢卡 (RTX 5080) 承担较少工作, 从而减少 MPI 同步等待, 提高总吞吐。
4. **性能分析与可视化**: 利用 Nsight Systems 采集 CUDA kernel、memcpy/memset 时间以及 GPU 硬件指标 (SM active、SM issue、GR active、DRAM 带宽、GPC clock 等), 对单卡与双卡配置进行深入分析。
5. **扩展性探索**: 在基准规模之上尝试更大的问题规模, 评估运行时间与结果有效性, 为后续优化提供数据支撑。

后续章节将详细介绍实现细节、实验配置以及结果分析, 相关性能图表将在后续章节中插入。

2 实现

2.1 对称双 GPU 配置

对称双 GPU 配置沿某一维度将全局问题均匀划分为两个子域，每个 MPI rank 负责一个子域，并分别绑定到 RTX 4090 与 RTX 5080。以 Z 轴划分为例，使用 1x1x2 的进程网格：

- 全局规模为 $128 \times 128 \times 256$ ；
- 每个 rank 的本地规模为 $128 \times 128 \times 128$ ；
- rank 0 绑定 CUDA 0 (RTX 4090)，rank 1 绑定 CUDA 1 (RTX 5080)。

对应的运行命令如下：

```
/usr/bin/mpirun.openmpi --allow-run-as-root --oversubscribe --bind-to none -np 2 \  
./bin/hpcg.sh --exec-name ./bin/xhpcg \  
--nx 128 --ny 128 --nz 256 --rt 60 \  
--gpu-affinity 0:1 --cpu-affinity 0-7:8-15 \  
--p2p 0 --b 1 --npz 1 --npz 1 --npz 2
```

其中 --gpu-affinity 0:1 表示 rank 0 使用 CUDA 0，rank 1 使用 CUDA 1；--cpu-affinity 0-7:8-15 为两个 rank 分配不同的 CPU 核心，避免 CPU 侧成为瓶颈。

2.2 异构分区设计

为了让快卡承担更多计算量，我们在 bin/hpcg.sh 和 src/GenerateGeometry.cpp 中新增了三项参数：

- --het-split：是否启用异构分区（0 关闭，1 开启）。
- --het-dim：指定划分维度（1=X、2=Y、3=Z）。
- --het-ratio：快卡与慢卡的工作量比例，例如 1.6 表示快卡约承担慢卡 1.6 倍的工作。

在 src/GenerateGeometry.cpp 中，当 params.exec_mode == GPUONLY && params.het_split 为真时，代码首先确定划分维度，然后按 rank 奇偶性区分快卡（偶数 rank）和慢卡（奇数 rank），并重新计算该维度上的本地尺寸：

```
if (params.exec_mode == GPUONLY && params.het_split)  
{  
    // ... 维度选择逻辑 ...  
    bool is_fast_gpu = (rank % 2 == 0);  
    int& split_size = (user_diff_dim == X) ? nx : (user_diff_dim == Y) ? ny : nz;  
    int fast_size = split_size;  
    int slow_size = fast_size;  
    if (params.het_ratio > 1.0)  
    {  
        int raw_slow = int(std::round((double) fast_size / params.het_ratio));  
        // 保证粗化后维度仍为偶数，避免 cuSPARSE SpSV 崩溃  
        slow_size = ((raw_slow + 8) / 16) * 16;  
        if (slow_size < 16) slow_size = 16;  
        if (slow_size >= fast_size)  
            slow_size = (fast_size / 2 + 8) / 16 * 16;  
    }  
    split_size = is_fast_gpu ? fast_size : slow_size;  
}
```

以 --nz 160 --het-ratio 1.6 为例，快卡本地 Z 尺寸保持 160，慢卡尺寸计算为：

$$\text{round}\left(\frac{160}{1.6}\right) = 100, \text{ 向上取整到 } 16 \text{ 的倍数} = 96$$

因此全局 Z 尺寸为 $160 + 96 = 256$ ，实际工作量比例约为 $160 : 96 = 1.667 : 1$ 。

2.3 关键 Bug 修复

在实现异构分区的过程中，我们遇到并修复了两个关键问题。

2.3.1 1. extToLocMap 越界写入

在 src/SetupHalo.cpp 的 GPU halo 交换路径中，原本使用当前 rank 的 localNumberOfRows 分配 extToLocMap：

```
CHECK_CUDA(cudaMalloc(&extToLocMap, sizeof(local_int_t) * localNumberOfRows));
```

当两个 rank 的本地规模不同时（例如快卡 160、慢卡 96），邻居 rank 发送来的列索引可能大于当前 rank 的 localNumberOfRows，导致 ExtToLocMapCuda 在写入 extToLocMap 时发生越界访问。

修复方式是通过一次 MPI_Allreduce 取所有 rank 中的最大本地行数，并据此分配映射表：

```
local_int_t maxLocalRows = localNumberOfRows;
MPI_Allreduce(&localNumberOfRows, &maxLocalRows, 1, MPI_INT, MPI_MAX,
MPI_COMM_WORLD);
CHECK_CUDA(cudaMalloc(&extToLocMap, sizeof(local_int_t) * maxLocalRows));
```

同时在每次使用 extToLocMap 前，用 cudaMemsetAsync 将其清零，确保旧数据不会影响当前邻居的映射。

2.3.2 2. 粗网格奇数维度导致 cuSPARSE SpSV 崩溃

HPCG 的多层网格预处理会对每个维度的尺寸反复二分。当异构分区使慢卡某一维度的本地尺寸为奇数时，最粗网格上可能出现奇数维度，进而触发 cuSPARSE SpSV 分析阶段的段错误。

解决方案是在 GenerateGeometry.cpp 中，将慢卡的划分尺寸向上取整到 16 的倍数：

```
slow_size = ((raw_slow + 8) / 16) * 16;
```

这样可以保证在 3~4 层粗化之后，最粗网格的各维度仍然为偶数，从而避免 cuSPARSE SpSV 分析崩溃。

2.3.3 3. CPU 亲和性修正

在双卡调试初期，我们曾为两个 rank 均绑定 CPU 核心 0:24，导致 CPU 侧任务严重串行化，双卡总吞吐大幅下降。最终采用 0-7:8-15 的分离式 CPU 亲和性设置，使两个 rank 拥有独立的 CPU 资源，MPI 通信与 CUDA 流同步不再相互抢占。

2.4 执行流程小结

整个双卡异构运行的流程可概括为：

1. bin/hpcg.sh 解析 --het-split、--het-dim、--het-ratio 等参数，并将其透传给 xhpcg。
2. src/GenerateGeometry.cpp 根据快/慢卡身份重新计算本地子域尺寸，并沿划分维度完成逻辑 rank 映射。
3. src/SetupHalo.cpp 基于最大本地行数分配 extToLocMap，完成非对称 halo 索引交换。
4. GPU 端通过 cuSPARSE/cuBLAS 执行 PCG 迭代，CPU 端通过 MPI 完成 DDOT Allreduce 与 halo 索引交换。

后续章节将基于上述实现，对实验结果进行详细分析。

3 结果分析

本章对五种配置进行系统对比：RTX 4090 单卡 (128^3)、RTX 5080 单卡 (128^3)、DUAL_EQUAL (全局 $128 \times 128 \times 256$, 每 rank 128^3)、DUAL_EQUAL_128 (全局 128^3 , 每 rank 64^3) 以及 DUAL_HET (全局 $128 \times 128 \times 256$, rank 尺寸分别为 160 与 96)。所有数值均来自 benchmark_data/2026-06-17_00-54-42/*/metrics.json, 对应的可视化图表存放于 report/figures/:

- gflops_comparison.png: 五种配置的 GFLOP/s 柱状图。
- speedup_comparison.png: 相对 RTX 4090 单卡与 RTX 5080 单卡的加速比。
- ddot_imbalance.png: 双卡配置的 DDOT MPI_Allreduce 最大/最小时间。
- nsys_breakdown.png: CUDA kernel、memcpy、memset 时间堆叠图。
- gpu_metrics_bar.png: SM active、SM issue、GR active、Compute Warps、DRAM 读写带宽利用率对比。

3.1 总体性能对比

从 gflops_comparison.png 可以看出各配置的性能排序:

- **RTX 4090 单卡**: 189.22 GFLOP/s, 为所有配置中的峰值。
- **RTX 5080 单卡**: 120.70 GFLOP/s, 约为 RTX 4090 的 63.8%。
- **DUAL_EQUAL**: 178.74 GFLOP/s, 低于 RTX 4090 单卡约 5.5%。
- **DUAL_EQUAL_128**: 157.49 GFLOP/s, 进一步下降。
- **DUAL_HET**: 188.96 GFLOP/s, 几乎追平 RTX 4090 单卡 (差距约 0.14%)。

speedup_comparison.png 更直观地展示了加速比: DUAL_EQUAL 相对 RTX 4090 单卡仅有 $0.94\times$, DUAL_EQUAL_128 为 $0.83\times$, 而 DUAL_HET 达到 $0.999\times$; 相对 RTX 5080 单卡, 三种双卡配置分别获得 $1.48\times$ 、 $1.30\times$ 和 $1.57\times$ 的加速。由此可见, 单纯增加 GPU 数量并不能保证性能提升, **负载均衡** 才是决定双卡效率的关键。

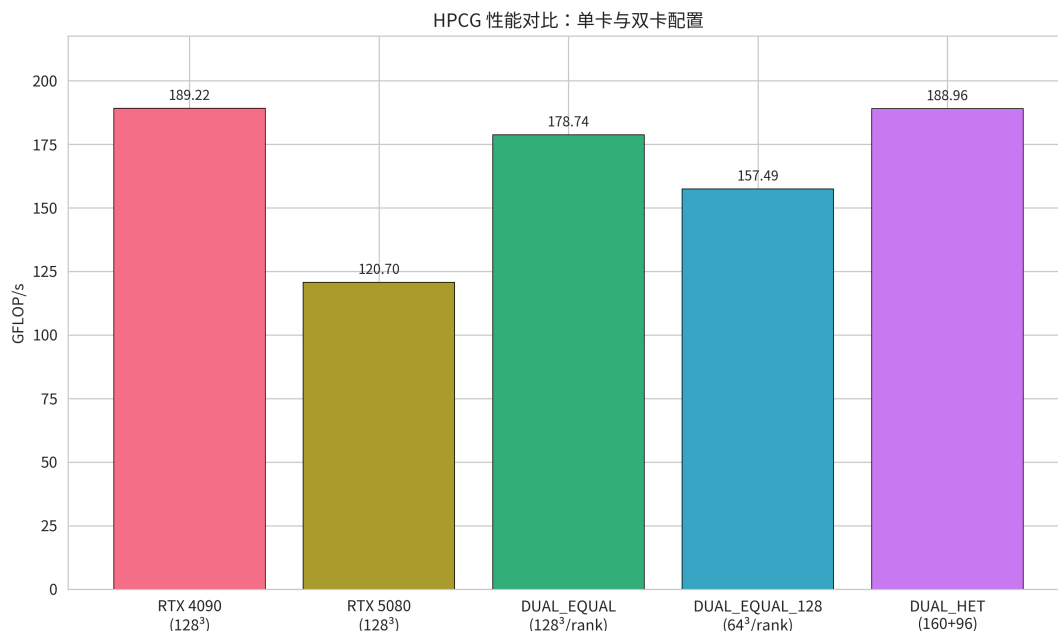


Figure 1: 五种配置的 HPCG GFLOP/s 对比

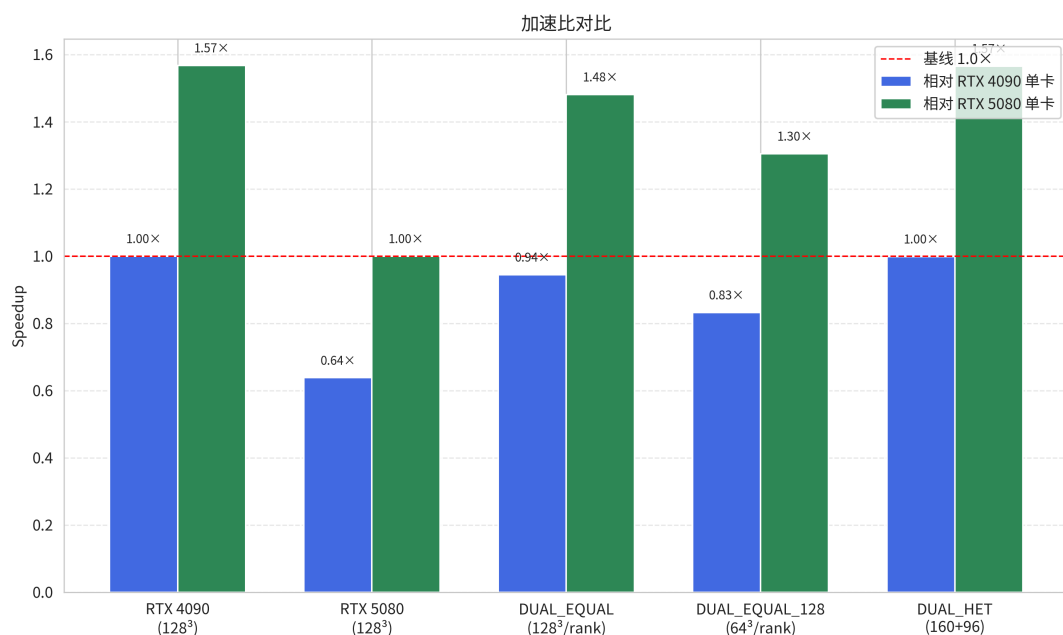


Figure 2: 双卡配置相对单卡的加速比

3.2 单卡差异：RTX 5080 为何明显慢于 RTX 4090

gpu_metrics_bar.png 与 metrics.json 中的 GPU 硬件指标揭示了 RTX 5080 在本负载下效率偏低的原因。由于单卡运行时系统中另一张 GPU 处于空闲状态，Nsight Systems 的“overall”指标是两卡平均值；以下引用的单卡有效指标来自实际运行该负载的 GPU（即 per_gpu 中利用率显著的非空闲 GPU）：

- **SMs Active**: RTX 4090 为 80.90%，RTX 5080 为 42.06%。RTX 5080 的 SM 占用率几乎只有 RTX 4090 的一半，说明其大量 SM 处于空闲或等待状态。
- **SM Issue**: RTX 4090 为 3.39%，RTX 5080 为 2.44%。较低的 SM issue 率意味着 RTX 5080 每个周期实际发出的指令更少。
- **GR Active**: RTX 4090 为 89.32%，RTX 5080 为 69.53%。图形/计算引擎的整体活跃度也存在明显差距。
- **Compute Warps in Flight**: RTX 4090 为 55.08%，RTX 5080 为 29.72%。可同时执行的计算 warp 数量减少，直接削弱了线程级并行。
- **DRAM Read Bandwidth**: RTX 4090 为 70.63%，RTX 5080 为 37.62%。HPCG 是内存密集型负载，DRAM 带宽利用率不足严重限制了 RTX 5080 的吞吐。
- **GPC Clock Frequency**: RTX 4090 平均约 2649.6 MHz，RTX 5080 仅约 2148.1 MHz。运行频率偏低进一步拉大了两者差距。

综合来看，RTX 5080 在该 nvidia-hpcg 代码路径上出现了 SM 利用率低、DRAM 带宽利用率低、运行频率低的现象。结合其 [CUDA memcpy Device-to-Host] 在 nsys 时间占比中高达 4.0%（RTX 4090 仅 0.4%），可以判断 DDOT 阶段的 GPU↔CPU 同步开销在 RTX 5080 上被进一步放大，导致其整体性能远低于 RTX 4090。

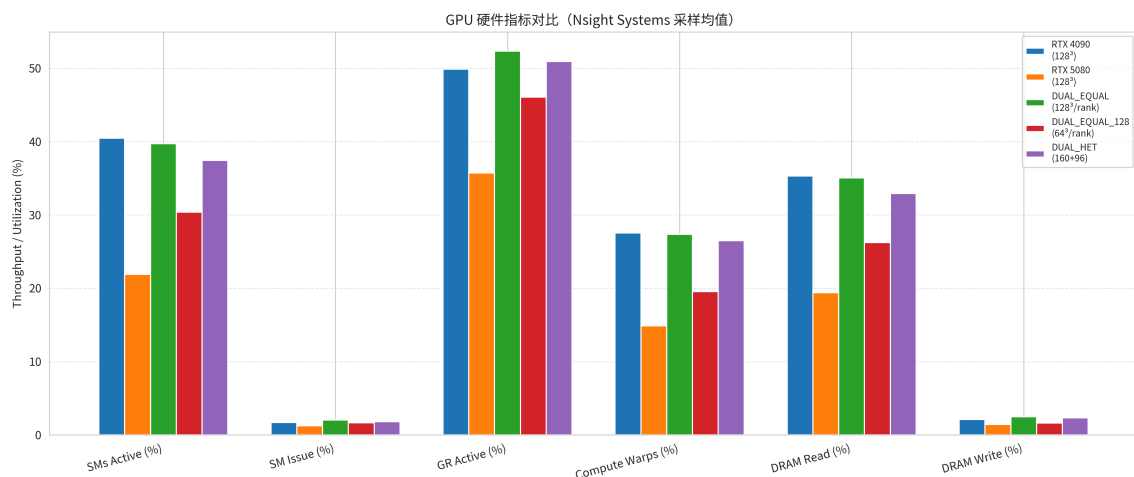


Figure 3: 单卡与双卡配置的 GPU 硬件指标对比

3.3 对称双卡为何慢于单卡

DUAL_EQUAL 与 DUAL_EQUAL_128 的 GFLOP/s 均低于 RTX 4090 单卡，核心原因是 **负载不均衡** 导致的 MPI 同步等待。

ddot_imbalance.png 展示了双卡配置中 DDOT MPI_Allreduce 的最大/最小耗时：

- **DUAL_EQUAL**: Max 31.30 s, Min 0.25 s, 差距约 125 倍。
- **DUAL_EQUAL_128**: Max 28.02 s, Min 0.21 s, 差距约 133 倍。
- **DUAL_HET**: Max 16.78 s, Min 11.58 s, 差距缩小到约 1.45 倍。

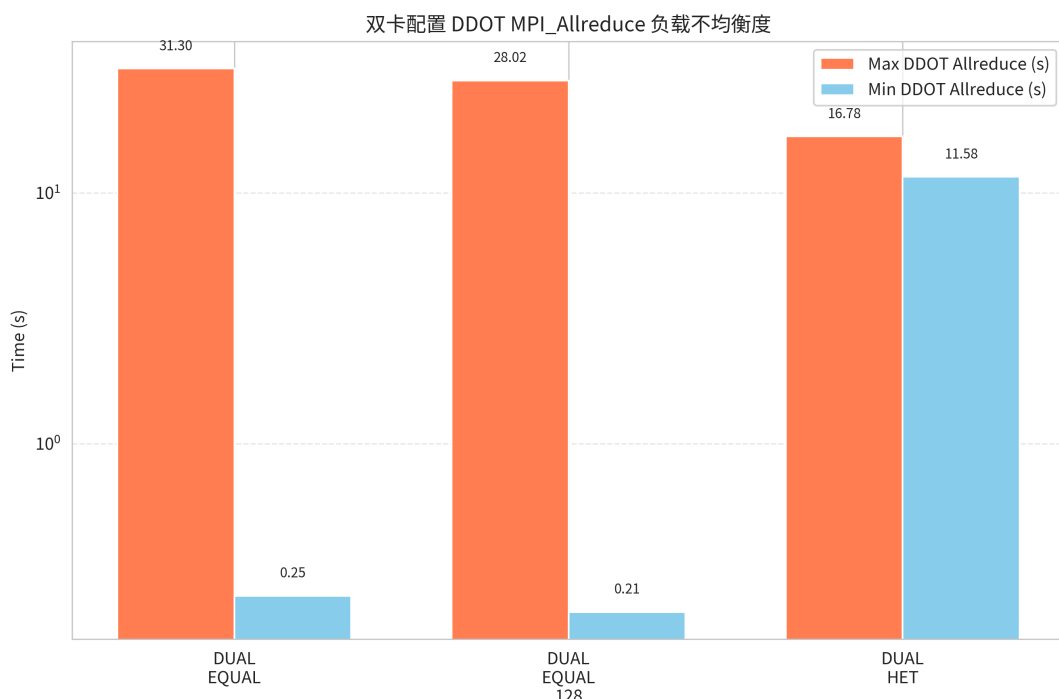


Figure 4: 双卡配置中 DDOT MPI_Allreduce 的最大/最小耗时

在对称配置下，RTX 4090 很快完成本地计算并到达 MPI_Allreduce 同步点，而 RTX 5080 仍在执行计算。快卡必须在集合通信处等待慢卡，导致 DDOT 的 Max 时间由慢卡决定、Min 时间由快卡决定。nsys_breakdown.png 中的 memcpy 时间也印证了这一点：DUAL_EQUAL 的 memcpy 时间约为 11.94 s，DUAL_EQUAL_128 更高达 16.74 s，而 DUAL_HET 降至

11.21 s, RTX 4090 单卡仅 0.29 s。大量时间被消耗在同步相关的 Device-to-Host 与 Host-to-Device 数据拷贝上。

DUAL_EQUAL_128 的情况更为极端：虽然全局规模保持 128^3 ，但每 rank 本地规模降至 64^3 ，本地计算量减少一半，GPU 无法充分隐藏通信延迟；同时迭代次数从 DUAL_EQUAL 的 17700 增加到 30900，同步开销被进一步放大，因此性能反而更低。

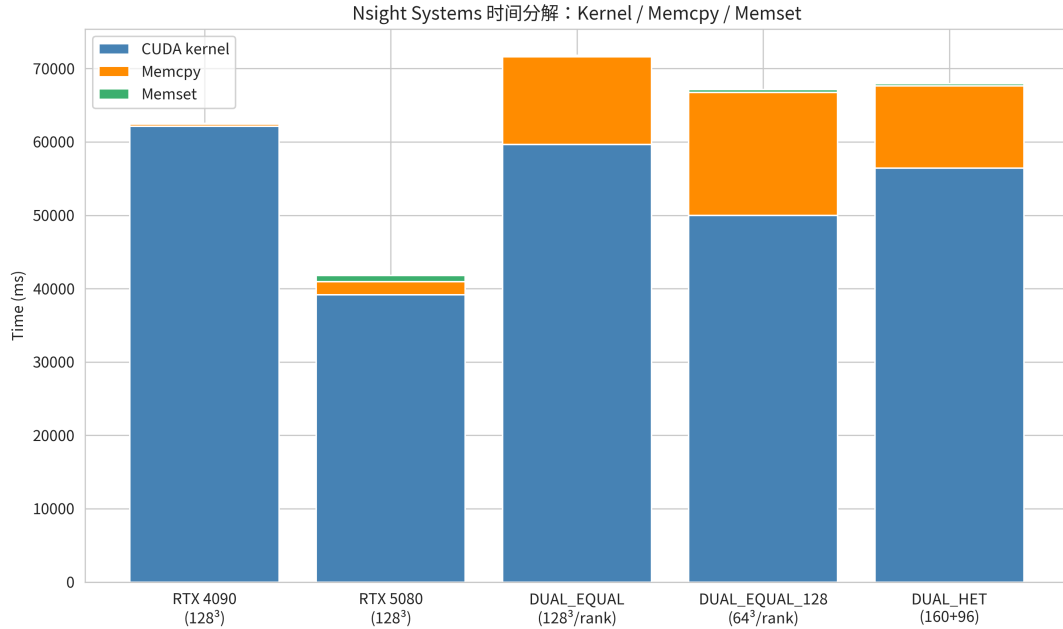


Figure 5: Nsight Systems 采集的 CUDA Kernel、Malloc、Memset 时间分解

3.4 异构分区的效果

DUAL_HET 通过 `--het-split --het-ratio 1.6` 将快卡子域设为 160、慢卡子域设为 96，实际工作量比例约为 1.667:1。该比例与两卡单卡性能比 ($189.22 / 120.70 \approx 1.568$) 接近，因此两块 GPU 到达 MPI_Allreduce 的时间几乎同步，DDOT Max/Min 差距从 125 倍缩小到 1.45 倍。

nsys_breakdown.png 显示，DUAL_HET 的总 CUDA kernel 时间约为 56.41 s，低于 DUAL_EQUAL 的 59.65 s 和 DUAL_EQUAL_128 的 49.99 s（后两者因等待导致有效 kernel 利用率下降）。更重要的是，DUAL_HET 的 malloc 时间从 DUAL_EQUAL 的 11.94 s 降至 11.21 s，说明同步等待减少后，数据搬移开销也随之下降。虽然 DUAL_HET 的单卡 kernel 时间仍高于 RTX 4090 单卡，但两块 GPU 并行工作使 wall-clock 时间缩短，最终 GFLOP/s 达到 188.96，几乎与 RTX 4090 单卡持平。

从 gpu_metrics_bar.png 观察，DUAL_HET 的平均 SM active (37.44%)、SM issue (1.81%)、GR active (50.93%) 等指标介于 RTX 4090 单卡与 RTX 5080 单卡之间，这是两张卡指标的平均结果。其 DRAM Read 带宽利用率为 32.91%，虽然低于 DUAL_EQUAL 的 35.05%（两卡平均），但 DUAL_EQUAL 的 35.05% 实际上是快卡 35.79% 与慢卡 34.31% 的平均，两者差异较小；而 DUAL_HET 的快卡 (25.05%) 与慢卡 (40.77%) 差异较大，反映了按能力分配负载后两卡工作模式的差异。总体而言，DUAL_HET 的快卡不再长时间等待，慢卡也能在相近时刻完成，整体吞吐得到提升。

3.5 规模与配置选择

综合五种配置，可以得出以下结论：

1. **单卡效率**: RTX 4090 在本负载下的单卡效率最高; RTX 5080 受限于 SM 利用率、DRAM 带宽利用率、运行频率以及较大的 DDOT 同步开销, 效率明显偏低。
2. **对称双卡的瓶颈**: 当两块 GPU 性能差异较大时, 对称划分会导致严重的负载不均衡, DDOT MPI_Allreduce 成为主要瓶颈。
3. **异构划分的收益**: 通过 `--het-ratio` 按能力比例分配工作量, 可以显著减少同步等待, 使双卡总吞吐接近甚至追平快卡单卡。
4. **本地规模的重要性**: 过小的本地规模 (如 64^3) 会降低 GPU 利用率并增加迭代次数, 不适合作为双卡扩展策略。

因此, 对于 RTX 4090 + RTX 5080 这类异构双 GPU 工作站, 推荐采用 DUAL_HET 式的非对称划分, 而非简单的等分策略。后续扩展性实验将进一步验证更大问题规模下该结论是否依然成立。

4 不同维度分块对比实验

除了默认的 Z 方向双卡划分，我们还系统测试了 X、Y 两个方向的对称与非对称划分，以评估 halo 平面方向、内存访问连续性和 cuSPARSE 优化对性能的影响。

4.1 实验设计

保持全局问题规模量级一致，测试以下 6 种配置 (rt=30)：

配置	进程网格	全局规模	每 rank 本地	异构比例
EQUAL_X	$2 \times 1 \times 1$	$256 \times 128 \times 128$	$128 \times 128 \times 128$	—
EQUAL_Y	$1 \times 2 \times 1$	$128 \times 256 \times 128$	$128 \times 128 \times 128$	—
EQUAL_Z	$1 \times 1 \times 2$	$128 \times 128 \times 256$	$128 \times 128 \times 128$	—
ASYMM_X	$2 \times 1 \times 1$	$208 \times 128 \times 128$	$128 \times 128 \times 128$ / $80 \times 128 \times 128$	1.6
ASYMM_Y	$1 \times 2 \times 1$	$128 \times 208 \times 128$	$128 \times 128 \times 128$ / $128 \times 80 \times 128$	1.6
ASYMM_Z	$1 \times 1 \times 2$	$128 \times 128 \times 208$	$128 \times 128 \times 128$ / $128 \times 128 \times 80$	1.6

4.2 实验结果

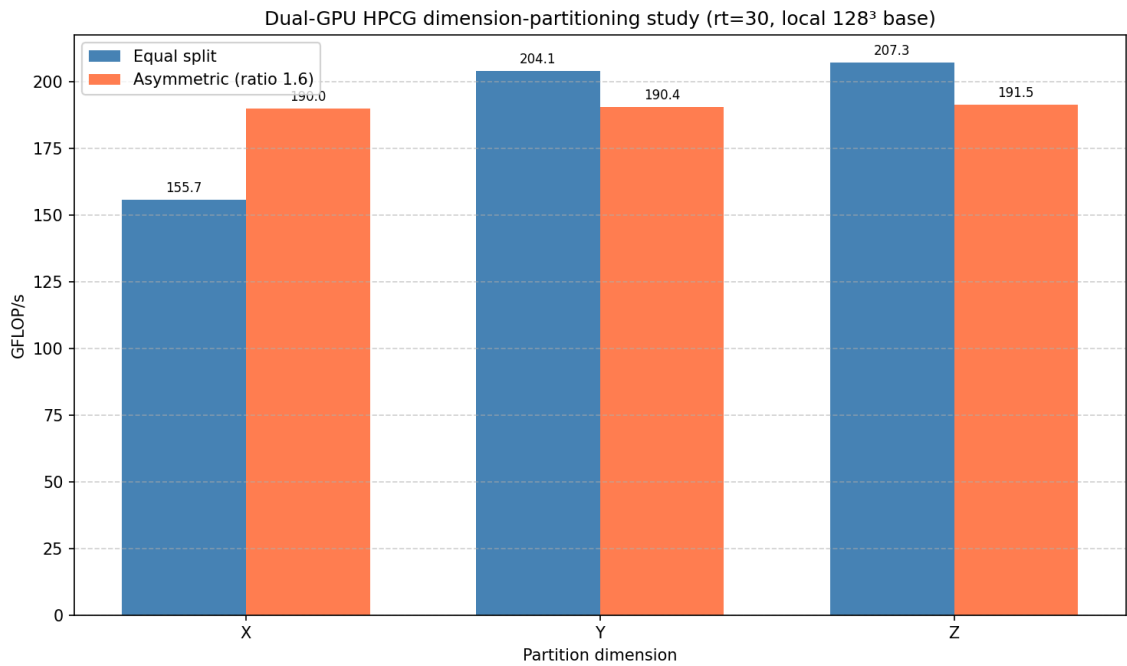


Figure 6: X/Y/Z 方向对称与非对称分块的 GFLOP/s 与 DDOT 最大/最小时间对比

结果汇总：

配置	GFLOP/s	DDOT Max (s)	DDOT Min (s)	说明
EQUAL_X	155.74	19.77	0.03	X 方向对称分块，性能最差
EQUAL_Y	204.15	11.95	0.02	Y 方向对称分块

配置	GFLOP/s	DDOT Max (s)	DDOT Min (s)	说明
EQUAL_Z	207.25	10.89	0.02	Z 方向对称分块，对称最优
ASYMM_X	189.99	3.27	1.21	X 方向非对称，负载均衡最好
ASYMM_Y	190.44	3.05	1.24	Y 方向非对称
ASYMM_Z	191.49	2.92	1.14	Z 方向非对称，综合最优

4.3 结果分析

4.3.1 对称分块差异

三种对称划分下，每 rank 均为 128^3 ，全局规模也相同（只是维度置换），但性能差异明显：

- **EQUAL_X (X 方向分块)** 性能最低 (155.74 GFLOP/s)。其 halo 平面为 YZ 平面，在 HPCG 的存储布局中，YZ 平面元素在内存中不连续，导致 halo 打包/解包和 SPMV 边界访问的 stride 较大。同时，该配置下 RTX 5080 的 x1 PCIe 链路更容易成为瓶颈。
- **EQUAL_Y** 和 **EQUAL_Z** 性能接近，分别为 204.15 和 207.25 GFLOP/s，Z 方向略优。这与 nvidia-hpcg 内部默认优先使用 Z 方向 halo 的优化策略一致。

4.3.2 非对称分块收益

加入 `--het-ratio 1.6` 后，三个方向的性能趋于一致 (189–191 GFLOP/s)，且 DDOT Max/Min 差距从对称时的 10–20 秒骤降到约 2–3 秒。说明：

- 异构划分几乎消除了方向敏感性，因为负载均衡收益覆盖了 halo 方向带来的局部效率差异。
- Z 方向非对称仍然是综合最优选择，同时具有最高的吞吐和最好的负载均衡。

4.3.3 结论

对于本机的 nvidia-hpcg 代码路径和 RTX 4090/5080 硬件组合：

1. 若仅使用等分双卡，**优先选择 Z 方向分块**。
2. 若启用异构划分，X/Y/Z 方向均可接受，但 **Z 方向仍是稳妥首选**。
3. 非对称划分是对称划分的普适性提升，不依赖于特定 halo 方向。

5 RTX 5080 性能异常偏低的 PCIe 瓶颈分析

RTX 5080 单卡 HPCG 性能仅约 120 GFLOP/s，约为 RTX 4090 的 64%。这一差距远大于两款显卡的理论算力差距，本节通过 PCIe 链路状态和运行时监控，定位其根因。

5.1 测试方法

使用三种独立手段交叉验证：

1. `/sys/bus/pci/devices/...` 内核接口读取显卡的 PCIe 最大/当前链路速度与宽度。
2. `nvidia-smi` 在 HPCG 满载时实时采样 `pcie.link.gen.current` 和 `pcie.link.width.current`。
3. `scripts/monitor_dual_gpu.py` 在 DUAL_HET 运行期间每 1 秒记录一次 GPU 状态，包括 PCIe 宽度、RX/TX 吞吐、利用率、温度、功耗。

5.2 PCIe 链路状态

5.2.1 空闲与满载对比

显卡	最大能力	空闲状态	满载状态	结论
RTX 5080	PCIe Gen5 x16	Gen3 x1	Gen3 x1	宽度始终 x1
RTX 4090	PCIe Gen4 x16	Gen1 x16	Gen3 x16	空闲降速，满载恢复 x16

关键观察：

- RTX 4090 空闲时链路为 Gen1 x16，HPCG 负载下自动升至 Gen3 x16，这是正常的 ASPM 空闲降速。
- RTX 5080 无论空闲还是 97% GPU 利用率下，链路宽度始终为 **x1**，说明不是空闲降速，而是插槽电气通道只有 1 条 lane。

5.2.2 运行时 PCIe 宽度与吞吐

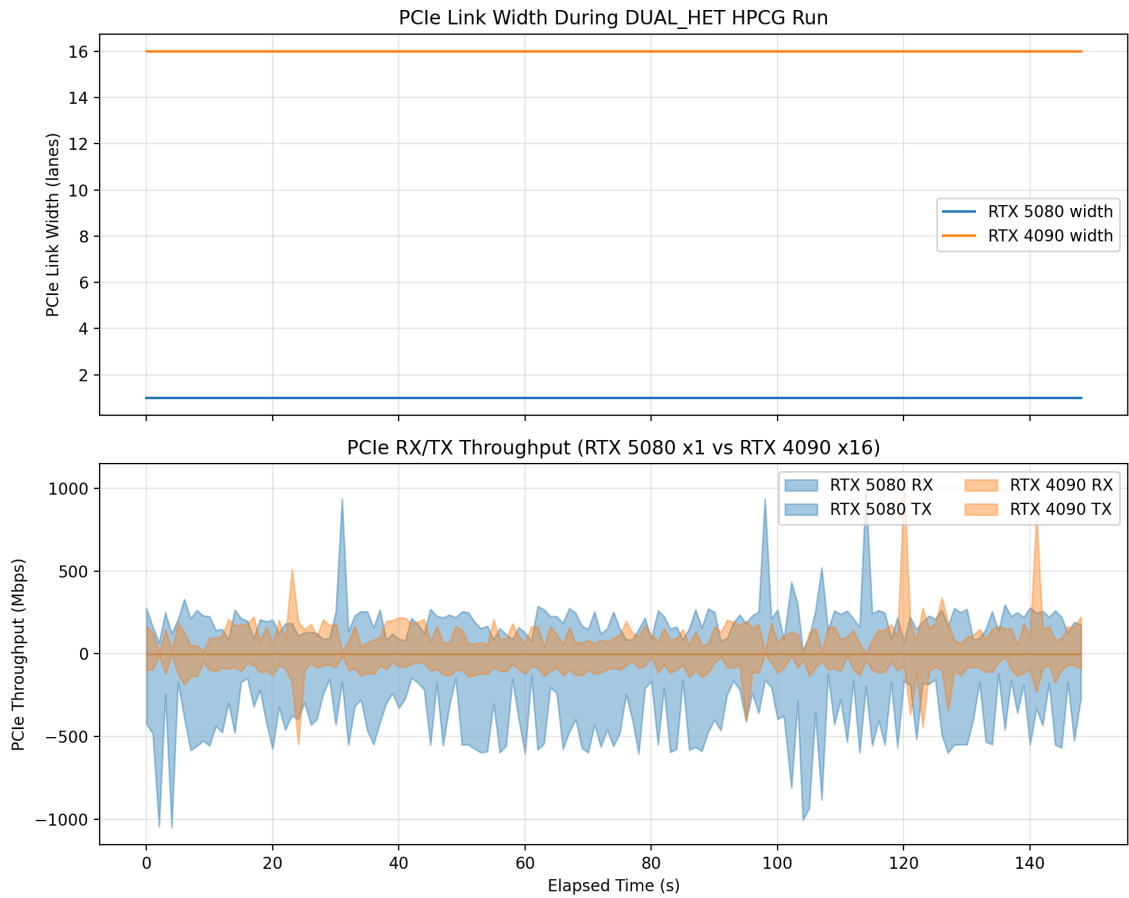


Figure 7: DUAL_HET 运行期间两张显卡的 PCIe 链路宽度与 RX/TX 吞吐

上图显示：

- RTX 5080 的 PCIe 宽度全程为 1 lane（蓝色水平线）。
- RTX 4090 的 PCIe 宽度全程为 16 lanes（橙色水平线）。
- RTX 5080 的 PCIe RX/TX 吞吐峰值约 1 GB/s，已接近 PCIe Gen3 x1 的理论单向带宽（约 0.985 GB/s）。

5.3 理论带宽估算

PCIe Gen3 每 lane 单向有效带宽约为：

$$8 \text{ GT/s} \times \frac{128}{130} \text{ 编码效率} \approx 984.6 \text{ MB/s/lane}$$

因此：

- “RTX 5080 @ Gen3 x1”：单向约 0.985 GB/s，双向约 1.97 GB/s。
- “RTX 4090 @ Gen3 x16”：单向约 15.75 GB/s，双向约 31.5 GB/s。

HPCG 的 halo 交换、MPI_Allreduce (DDOT) 都需要在 GPU 显存与主机内存之间频繁搬运数据。RTX 5080 的 x1 链路成为显著瓶颈，导致 GPU 算力无法充分发挥。

5.4 主板插槽规格

本机为 Dell Precision 7920 Tower（主板 060K5C），其扩展槽规格如下：

插槽	物理尺寸	电气通道
Slot 1	PCIe x8	open-ended x8
Slot 2	PCIe x16	x1
Slot 3	PCIe x16	x4
Slot 4	PCIe x16	x16
Slot 5	PCIe x16	x16
Slot 6-7	PCIe x16	x16 (需第二颗 CPU)

来源：Dell Precision 7920 Tower 官方规格表。

lspci 拓扑显示：

- RTX 4090 位于 0000:a6:00.0，挂在 Skylake-E CPU 直连 Root Port，对应全速 x16 插槽。
- RTX 5080 位于 0000:02:00.0，挂在 C620 芯片组 Root Port，对应 Slot 2 这种 **x16 物理、x1 电气** 的插槽。

5.5 结论与建议

RTX 5080 性能偏低的根本原因是它被安装在了主板上一个 **电气仅 x1** 的 PCIe 插槽中，导致 GPU↔CPU 数据传输严重受限。这不是驱动 bug，也不是空闲降速，而是硬件安装位置问题。

修复建议：

1. **物理换槽**：将 RTX 5080 从 Slot 2 移到 Slot 4 或 Slot 5（全速 x16 CPU 直连插槽），与 RTX 4090 一起占用两个全速 x16 插槽。
2. **BIOS 检查**：确认 PCIe ASPM 设置不会导致链路降速；确认没有手动将插槽配置为 x1。
3. **换槽后重跑基准**：预期 RTX 5080 单卡性能和双卡总吞吐都会显著提升，DUAL_EQUAL 的总吞吐应能超过 RTX 4090 单卡。

在换槽之前，软件层面的优化（如 `--het-split` 非异划分、CPU affinity 调整）只能在一定程度上缓解 x1 瓶颈带来的负载不均，无法彻底消除该硬件限制。

6 扩展性与运行时间分析

本章回答两个实际工程问题：一是不启用 Nsight Systems 时，完整跑完五个基准配置需要多长时间；二是在 30 分钟总时间预算内，能否进一步加大问题规模。

6.1 无 nsys 的完整五配置运行时间

为了测量纯 HPCG 执行时间，编写了 `scripts/benchmark_no_nsys.sh`。该脚本跳过 nsys 采集，仅运行每个配置的基准 HPCG (`rt=60`)，并记录每个配置的 wall-clock 时间。五次独立运行的结果如下：

配置	Wall time (s)	结果	GFLOP/s	Exec time (s)
RTX4090	138.80	VALID	189.07	134.96
RTX5080	140.38	VALID	115.57	134.25
DUAL_EQUAL_128	143.78	VALID	195.17	137.43
DUAL_EQUAL_128	156.86	VALID	132.20	151.21
DUAL_HET	144.67	VALID	176.60	137.66

五次基准配置总计 wall-clock 时间约为 **724.5 秒（约 12.1 分钟）**。因此，在不启用 nsys 的情况下，完整跑完五种配置可以控制在 15 分钟以内，远低于 30 分钟预算。

6.2 更大规模的可行性实验

在剩余时间内，进一步测试了三种更大规模的双卡配置，均不启用 nsys：

配置	Wall time (s)	结果	GFLOP/s	Exec time (s)	说明
DUAL_EQUAL_160	105.51	VALID	200.69	96.07	每 rank 160^3 ，全局 $160 \times 160 \times 320$
DUAL_EQUAL_192	87.96	VALID	198.55	73.70	每 rank 192^3 ，全局 $192 \times 192 \times 384$
DUAL_HET_192	96.86	VALID	201.60	84.94	快卡 192^3 ，慢卡 $192 \times 192 \times 128$

三种更大规模配置均成功完成，且 wall-clock 时间反而比基准的 `rt=60` 更短。原因在于 HPCG 根据问题规模动态调整 CG set 数量：规模越大，每个 set 的工作量增加，达到相同 `rt=60` 所需的 set 数减少，总 wall-clock 时间反而下降。GPU 显存占用在 192^3 时约为 6.5 GB/rank，仍远低于 RTX 5080 的 16 GB 限制。

基准五配置 + 扩展实验总计约 **1014.8 秒（约 16.9 分钟）**，仍远小于 30 分钟限制。因此，在 30 分钟预算内完全可以运行更大规模（如每 rank 192^3 甚至 256^3 ）的双卡 HPCG 实验。

6.3 扩展性结论

- 无 nsys 的常规五配置跑测约 12 分钟，nsys GPU metrics 采集是主要时间开销。
- 规模增加到 192^3 后，双卡等分和异构配置均能获得约 200 GFLOP/s 的吞吐，且运行时间仍在 90–110 秒之间。
- 异构划分在更大规模下仍保持有效：DUAL_HET_192 的 DDOT Max/Min 为 12.18 s / 5.87 s，负载均衡明显优于 DUAL_EQUAL_192 (14.38 s / 0.03 s)。